MSCS-532

Algorithms and Data Structures

Spring 2026

Name: Victor Hugo Abad Navarrete

Assignment 3: Understanding Algorithm Efficiency and Scalability

University of the Cumberlands

1. Theoretical Analysis

This section provides a theoretical analysis of randomized quick sort and hashing with chaining algorithms.

1.1 Randomized Quick Sort

A quicksort algorithm is a divide-and-conquer method that has the objective of sorting an array by dividing it into two subarrays in each recursive step, and rearranging them to make sure that each of the subarrays is either greater or lower than a chosen pivot point. In the divide step, the array is partitioned into two subarrays based on a pivot. In this step, the two subarrays are created by making sure that in one, all the data points are less than or equal to the pivot point, and in the other one, they are greater than the pivot point. In the conquer step, quicksort is called recursively for each of the subarrays. In the combine step, all the subarrays are merged to form the final sorted array[1]. The performance of quicksort is dependent on the pivot chosen in each step. It can produce subarrays that either contain all the elements of the original array, or 0 elements in the extreme cases. The randomized quick sort algorithm chooses the pivot point at random.

Average Case Analysis

The total running time of the quicksort algorithm can be expressed as the sum of the cost of the total number of recursive calls and the total time spent across all the recursive functions. The total number of recursive calls is upper-bounder by n. This is the case where all the pivots are chosen such that one of the arrays contains all the elements minus the pivot and the other contains the pivot. If the time spent on each of the recursive function is X, the expected value E[X], can be expressed as:

$$E[X] = O(n) + f(X)$$

Where f(x) is the total time spent across all the recursive functions. Using probability analysis, f(x) can be thought as the sum of the probabilities that two arbitrary elements i,j (i<j) are compared in the array. This probability is 2/(j -i +1). This follows from the fact that the probability that two elements are compared is the probability that either is chosen as the first pivot in the array (Cormen, Leiserson, & Rivest, 2022). The probability is assumed to be uniform so all elements have the same probability of being chosen.

$$\Pr\{z_i \; is \; compared \; with \; z_j\} = \frac{2}{j - i + 1}$$

Each pair of i,j is compared at most once. Then f(x) can be expressed as:

$$f(x) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E[X_{ij}]$$

$$f(x) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr\{z_i \; is \; compared \; with \; z_j\}$$

$$f(x) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j - i + 1}$$

If change is variables is used (k = j -1):

$$f(x) = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k + 1}$$

Since n – i > n, and k + 1> k:

$$f(x) < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k}$$

$$f(x) = \sum_{i=1}^{n-1} O(\lg n)$$

$$f(x) = O\ (n \lg n)$$

(Cormen, Leiserson, & Rivest, 2022)

$$E[X] = O(n) + O\ (n \lg n)$$

$$E[X] = O\ (n \lg n)$$

1.2 Hashing with Chaining

Hashing with chaining is a technique used in the creation of dictionaries that use a key as the lookup method. The translation from a key to an array index follows a hashing function that maps n total keys into m array entries (slots). In the simple uniform hashing assumption, each key is equally likely to be hashed to any slot in the array, regardless of the other keys. Hashing with chaining is special in how it handles hashing conflicts, that is, when multiple keys map to the same array index. In hashing with chaining, if multiple keys map to the same array entry, they are all added to that entry. The load factor is defined as the ratio of all the keys (n) to the number of slots (m).

$$\alpha = \frac{n}{m}$$

By the uniform hashing assumption, this corresponds to the number of expected elements in each slot.

Search function

The search function is the sum of the time required to hash the key and produce the array index, plus the time required to search for the key in the linked list that exists in the slot that is mapped to the key. Since the number of expected elements on each slot is expressed by the load factor, the search function is bounded by:

$$S = c1 + c2\ \alpha$$

$$S = O(1 + \ \alpha)$$

For the fraction n/m, the only scenario in which alpha renders a non-constant complexity is when n >> m. Hence, the requirement to obtain a linear search complexity is:

$$m = \Omega(n)$$

This is a lower bound, but increasing m at a much higher rate than n would just increase the memory footprint of the array while not providing any extra benefit. Therefore, ideally, m would be upper and lower bounded by n:

$$m = \Theta(n)$$

Insert function

The insert function first hashes the key and then traverses the linked list in the slot to check for duplicate keys. This is the same cost as the search function so the same analysis also applies to insert, and the bounds are:

$$I = O(1 + \ \alpha)$$

Once again, a correct balance of complexity and memory usage is given by m=Θ(n).

Delete function

The delete function has the same cost as the search and insert functions. The total cost is the cost of hashing plus the cost to search the linked list for the matching element.

$$D = O(1 + \ \alpha)$$

For delete, then the balance of complexity and memory usage is also given by m=Θ(n).

Load factor

As described by the analysis of the search, insert, and delete functions, the load factor determines the complexity of the three functions. A good load factor would remain constant, given that m increases proportionally to n. One of the strategies to maintain a low load factor is the creation of a large enough array (m) that can accommodate the anticipated number of keys. Another strategy consists on the dynamic resizing of the array (entry table). In programming languages where arrays are expected to be of static size, an array of size m can be allocated whenever n>km. Similarly, as n <<m, one of the extra arrays that were previously created can be deleted to save memory. This imposes one requirement on the hashing function, that is, the hashing function should be designed to produce the same value regardless of the number of elements in the array. An additional mapping function would then map the result of the hash function to an index, based on the number of subarrays.

2. Implementation and comparison

The randomized quick sort and hashing with chaining algorithms were implemented in Python in the following repository:

https://github.com/victorvic-ucumberlands/MSCS532_Assignment3

To test the performance of randomized quick sort, random datasets of 100, 1000, 10000, 100000, 1000000, and 10000000 were created. Also, four arrays were created for each of the data sizes. In the first one, the data was kept random, as it was generated. In the second one, the data was sorted in ascending order (the same as the test algorithm). In the third one, the data was sorted in descending order. In the fourth one, a new randomized array with a much more limited randomization space was created for each of the data sizes. The reduced randomization space

meant the creation of arrays with many repeated elements. Figures 1 and 2 show the time taken to sort the array and the peak memory consumption, respectively.

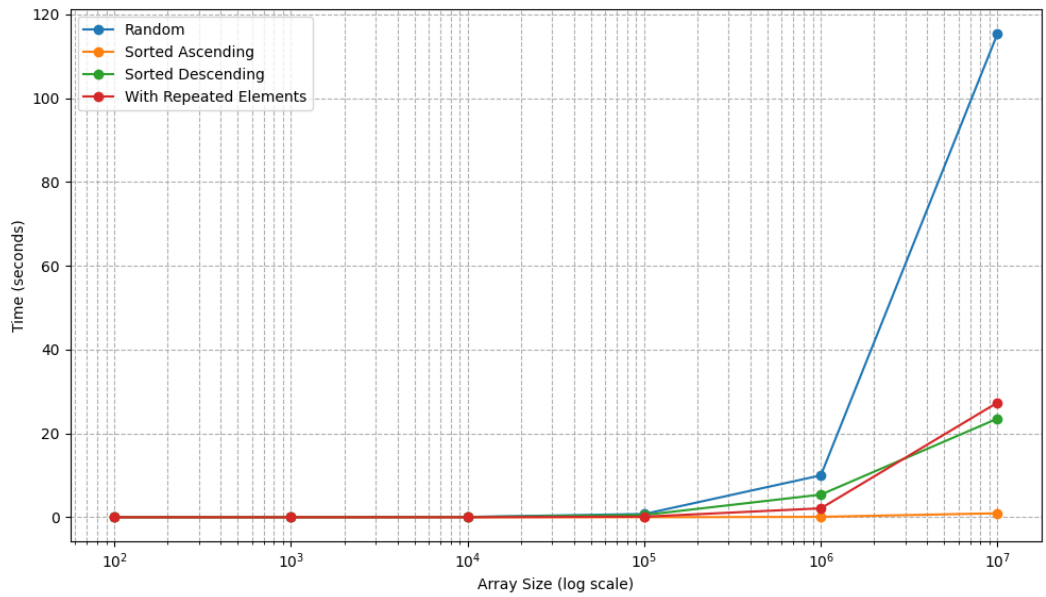

Figure 1: Performance of the quick sort algorithm with different data sizes and array types.

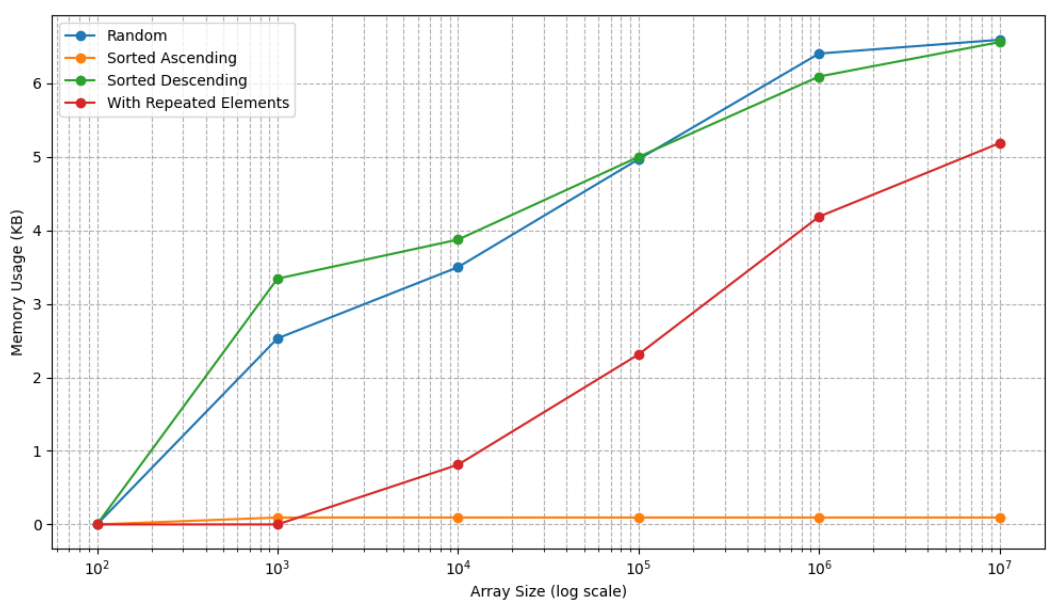

Figure 2: Memory consumption of the randomized quick sort algorithm with different data sizes and array types

This implementation reveals interesting points about quicksort and the results obtained when corner cases are handled appropriately. First, when supplying a totally random array to randomized quick sort, the algorithm complexity is approximately square. This might be originating from two reasons. First, generating a random number in the array length space incurs additional overhead. Depending on the guarantees of the pseudo-random number generator and the system architecture, generating this random number could be costly. Second, when trying to improve performance for already-sorted arrays or subarrays, the entire array or subarray is traversed an additional time to check whether the base case has been reached. This cost is added to every recursive call and can be significant for very deep ones. Also, since this array is random and every subarray will be, the new base case (an already sorted array) is unlikely to be reached.

For a totally random array, the performance of the randomized quick sort algorithm is bound by $O(n^2)$. However, Figure 1 shows very good performance for already sorted arrays, arrays in reversed order, and arrays with many repeated elements. For already-sorted arrays, the cost is linear, since the base case is reached in the first call to the function. For the case of a reverse-sorted array, because the first half of the array will be placed in reverse order starting from the end of the array, the subarrays will be sorted very quickly. Because of the enhancement that added the already sorted array/subarray to the base case, the recursive calls will detect this and return, ending the recursive step very quickly. For arrays with repeated elements, modifying the array partitioning when the pivot has multiple copies improved performance. Partitioning is guaranteed to be balanced with respect to the pivot elements. Also, since fewer element values exist in each subarray, it is more likely that they will be sorted by the partitioning steps, and the newly added base case (a subarray already sorted) will be reached more often, ending the recursion faster. In

summary, this implementation of randomized quick sort provides a real-world bound of $O(n^2)$ and handles the corner cases very well. The addition of a new base case for already sorted arrays improves the performance in various corner cases, and the rebalancing of partitioning for arrays with many repeated elements improves the performance for this kind of arrays.

References

Cormen, T., Leiserson, C., & Rivest, R. (2022). *Introduction to Algorithms* . Cambridge: The MIT Press.